Giddy Protocol Exploit Analysis

researcher : ali.k maki

Date: 27 april 2026

How did the Giddy Protocol hack happen? -Full Hack Analysis

1- Affected Components

The attack targeted the Smart Vault system, specifically the implementation contract: 0x5f0ad32c00641d1d2bb628ff341e0d4bb4494318 This contract is responsible for managing assets (tBTC, cbBTC, WBTC) through Proxies.

2- The Vulnerability: Incomplete Signature

The protocol relies on a digital signature issued by the company's backend to authorize swap operations. The programmatic issue is that the signature did not "bind" (Bind) all data together.

What was verified? When calling the `compound()` function, the contract verifies the signature inside the internal function `_validateAuthorization`. The signature guaranteed the validity of:

- Raw swap data (`swap.data`).
- Transaction sequence number (`nonce`).
- The total authorized amount.

What did the developers leave unprotected? There are programmatic fields called "Wrapper fields" that are passed with the signature but are not part of it, the most important being:

1- Destination Address (`aggregator`): The contract that will receive the authorization to execute the swap.

2- Allowance Value (`amount`): The amount the contract allows the external party to withdraw.

3- Attack Sequence

Step One: Signature Interception The attacker obtained a valid signature (Valid Signature) that had not yet been executed. This signature carries the "seal of trust" from the protocol's backend.

Step Two: Re-crafting the Transaction Since the destination address (`aggregator`) is not protected by the signature, the attacker did the following:

- Placed their own malicious contract address in the destination field instead of trusted addresses (such as Uniswap).
- Placed the value `MAX_UINT256` (the largest possible number in programming) in the allowance field.

Step Three: Deceiving the Strategy When the smart contract received the transaction:

1. It checked the signature and found it "valid" (because the attacker did not change the signed data).
2. Based on this signature, the contract executed the `approve` function (granting withdrawal permission) to the address placed by the attacker and for the infinite amount they specified.

Step Four: Asset Draining (The Drain) Once the attacker obtained open withdrawal permission from the vault, they no longer needed the signature. They called the withdrawal function from their malicious contract to drain the liquidity tokens (Gauge Tokens) and transfer them to their private wallet, exploiting the permission obtained in the previous step.

4- Technical Conclusion

The attack was not a breach of encryption, but rather an exploitation of the signature being partial. The attacker used an "official authorization" to execute an "unofficial order." The smart contract trusted the signature but did not verify that this signature was designated for the entity requesting the funds.

Advice: Always remember that a digital signature in smart contracts must include **(Who, What, and To Where)**. If any of these pillars fall, an attacker can redirect the transaction to their advantage.